

RAM CHARAN VANGA

RA2511027010164

Sub ject : DSA

Unit 3, S5, SLO2 - Stack Applications - Evaluating Postfix Expression

Algorithm to evaluate a postfix expression

Answer: Algorithm EvaluatePostfix(exp):

Step 1: Create an empty Stack.

Step 2: Scan the Postfix expression from left to right token by token.

Step 3: For each scanned element X:

a. If X is an operand (number), push X onto the Stack.

b. If X is an operator (such as +, -, *, /, %):

i. Pop operand2 from Stack: operand2 = pop()

ii. Pop operand1 from Stack: operand1 = pop()

iii. Compute result = operand1 (operator) operand2

iv. Push result back onto the Stack.

Step 4: When the expression ends, the final result is the only value remaining on the Stack: return pop().

Step 5: Exit.

Follow the Algorithm and Solve: Evaluation of a postfix expression

9 3 4 * 8 + 4 / -

Answer: Step-by-step Evaluation Trace:

1. Scan '9': Operand -> Push 9 -> Stack: [9]

2. Scan '3': Operand -> Push 3 -> Stack: [9, 3]

3. Scan '4': Operand -> Push 4 -> Stack: [9, 3, 4]

4. Scan '*': Operator -> Pop 4, Pop 3 -> Compute $3 * 4 = 12$ -> Push 12 -> Stack: [9, 12]

5. Scan '8': Operand -> Push 8 -> Stack: [9, 12, 8]

6. Scan '+': Operator -> Pop 8, Pop 12 -> Compute $12 + 8 = 20$ -> Push 20 -> Stack: [9, 20]

7. Scan '4': Operand -> Push 4 -> Stack: [9, 20, 4]

8. Scan '/': Operator -> Pop 4, Pop 20 -> Compute $20 / 4 = 5$ -> Push 5 -> Stack: [9, 5]

9. Scan '-': Operator -> Pop 5, Pop 9 -> Compute $9 - 5 = 4$ -> Push 4 -> Stack: [4]

End of expression reached.

Final Evaluated Result = 4